

Week 2: Intelligent Agents

1. Introduction to Intelligent Agents

Artificial Intelligence is not only about machines “thinking.”

A big part of AI is designing systems that can **observe**, **decide**, and **act**.

Such a system is called an **agent**.

Definition of an Agent

An **agent** is anything that can:

1. perceive its environment through **sensors**
2. act upon that environment through **actuators**

In simple words, an agent takes input from the environment, processes it, and produces an action.

Definition of an Intelligent Agent

An **intelligent agent** is an agent that acts **rationally** to achieve a goal.

A rational agent chooses the action that is most likely to produce the best result.

2. Basic Structure of an Agent

The general structure of an intelligent agent is:

Environment → **Sensors** → **Agent** → **Actuators** → **Environment**

Explanation

- ☺ **Environment**: the world in which the agent operates
- ☺ **Sensors**: collect information from the environment

- ☺ **Agent:** makes decisions
- ☺ **Actuators:** perform actions in the environment

Example 1: Human as an Agent

- ☺ Sensors: eyes, ears, nose, skin
- ☺ Actuators: hands, legs, mouth
- ☺ Environment: surroundings

Example 2: Robot Vacuum Cleaner

- ☹ Sensors: obstacle sensors, dirt sensors
- ☹ Actuators: wheels, brushes
- ☹ Environment: room or house

Example 3: Self-Driving Car

- ☹ Sensors: cameras, radar, lidar, GPS
- ☹ Actuators: steering, brakes, accelerator
- ☹ Environment: roads, traffic, pedestrians

3. Rationality in Agents

An agent is called **rational** if it does the right thing based on:

- ☹ the available information
- ☹ the actions it can perform
- ☹ the goal it wants to achieve

This does not mean the agent is always perfect.

It means the agent selects the best possible action according to what it currently knows.

Example

If a delivery robot sees an obstacle in front of it, a rational action is to stop or change direction instead of moving forward and crashing.

4. Types of Agents

There are several types of intelligent agents. These types differ in how they make decisions.

4.1 Simple Reflex Agent

A **simple reflex agent** selects actions based only on the **current percept**.

It does not remember past events.

It follows condition-action rules.

General Rule

If condition, then action

Example

- ⊗ If room is dirty → clean
- ⊗ If obstacle ahead → turn left
- ⊗ If temperature is high → turn on fan

Characteristics

- ☺ very simple
- ☺ fast decision-making
- ☺ works well in simple environments
- ☺ fails when the environment is complex

Real-Life Example

Automatic door:

- If a person is detected → open door
- Otherwise → keep closed

Limitation

It cannot handle situations that require memory or planning.

4.2 Model-Based Reflex Agent

A **model-based reflex agent** keeps track of the internal state of the environment.

It uses:

- ☺ current percept
- ☺ previous percepts
- ☺ internal model of the world (**Memory + Rules about how the environment works**)

Why it is needed

Sometimes the agent cannot see the full environment at one time.

So it must maintain an internal representation.

Example

A robot moving in a room may remember:

- where obstacles were seen
- where it has already cleaned

Characteristics

- better than simple reflex agent
- can work in partially observable environments
- uses memory

Example

A robot vacuum may remember which rooms are already cleaned.

4.3 Goal-Based Agent

A **goal-based agent** takes actions to achieve a specific goal.

It does not only react — it also considers the future.

Example

Google Maps finding a route:

- ✖ current state: current location
- ✖ goal: destination
- ✖ action: choose the best path

Characteristics

- more flexible
- can compare multiple paths or options
- requires search and planning

Example in AI

In chess:

- ❖ current board state
- ❖ goal: win the game
- ❖ agent chooses moves that improve its chances

4.4 Utility-Based Agent

A **utility-based agent** chooses the action that provides the **highest utility**.

Utility means a numerical measure of how good or desirable a state is.

Why utility is useful

Sometimes there are many ways to reach a goal.

The agent needs to choose the best one.

Example

A self-driving car wants to reach the destination, but there are many route options:

- ✓ shortest route
- ✓ safest route
- ✓ cheapest route
- ✓ fastest route

A utility function helps choose the most preferred option.

Characteristics

- more advanced than goal-based agents
- can handle trade-offs
- useful in uncertain environments

Example

An online shopping recommendation system may rank products based on:

- ☺ user interest
- ☺ price
- ☺ popularity
- ☺ rating

4.5 Learning Agent

A **learning agent** improves its performance over time by learning from experience.

This is one of the most important types in modern AI.

Main Idea

The agent starts with limited knowledge and becomes better after observing results.

Components of a Learning Agent

A learning agent usually has four parts:

1. Performance element

Chooses external actions

2. Learning element

Improves the performance element

3. Critic

Gives feedback on performance

4. Problem generator

Suggests exploratory actions to learn more

Example

A spam email filter:

- ☺ initially makes mistakes
- ☺ learns from labeled examples
- ☺ improves classification over time

Real-Life Examples

- ☺ Netflix recommendations
- ☺ YouTube suggestions
- ☺ fraud detection systems
- ☺ chatbots that improve with training data

5. Comparison of Agent Types

Agent Type	Basis of Action	Memory	Goal-Oriented	Learning
Simple Reflex	Current percept only	No	No	No
Model-Based Reflex	Current percept + internal state	Yes	No	No
Goal-Based	Goal + future consequences	Yes	Yes	No
Utility-Based	Utility/value of outcomes	Yes	Yes	No

Learning Agent	Experience and feedback	Yes	Can be	Yes
----------------	-------------------------	-----	--------	-----

6. PEAS Description

PEAS is a framework used to describe an intelligent agent's task environment.

PEAS stands for:

- ❖ P = Performance Measure
- ❖ E = Environment
- ❖ A = Actuators
- ❖ S = Sensors

This helps us clearly define what an agent does.

6.1 Performance Measure

This tells us how success is measured.

Examples:

- accuracy
- speed
- safety
- cost
- efficiency

6.2 Environment

The environment is the world where the agent operates.

Examples:

- road for a self-driving car
- hospital for a medical diagnosis system
- chessboard for a chess agent

6.3 Actuators

Actuators are the components used to take action.

Examples:

- ♦ wheels
- ♦ robotic arms
- ♦ speakers
- ♦ brakes
- ♦ steering system

6.4 Sensors

Sensors gather information from the environment.

Examples:

- ✕ camera
- ✕ microphone
- ✕ keyboard input
- ✕ GPS
- ✕ radar

7. PEAS Examples

Example 1: Self-Driving Car

Performance Measure

- safety
- speed
- fuel efficiency
- passenger comfort
- obeying traffic laws

Environment

- roads
- traffic signals
- vehicles
- pedestrians
- weather conditions

Actuators

- steering
- brakes
- accelerator
- horn
- display signals

Sensors

- camera
- radar
- lidar
- GPS
- speedometer

Example 2: Vacuum Cleaning Robot

Performance Measure

- ★ maximum dirt cleaned
- ★ minimum time
- ★ low energy consumption
- ★ avoid collisions

Environment

- ★ rooms
- ★ walls
- ★ furniture

- ★ dirt
- ★ stairs

Actuators

- ★ wheels
- ★ vacuum motor
- ★ brushes

Sensors

- ★ dirt sensor
- ★ bump sensor
- ★ distance sensor
- ★ camera

Example 3: Medical Diagnosis System

Performance Measure

- diagnostic accuracy
- low error rate
- timely response

Environment

- patient records
- symptoms
- medical tests
- disease database

Actuators

- screen display
- printed report
- alert message

Sensors

- input forms
- medical test data
- patient history database

Example 4: Chess Playing Agent

Performance Measure

- win the game
- avoid losing
- minimize mistakes

Environment

- chessboard
- opponent moves

Actuators

- ☺ moving chess pieces on screen or robot arm

Sensors

- ☺ board state input
- ☺ opponent move detection

8. Environment Types

Not all environments are the same.

Different AI agents work in different kinds of environments.

Understanding the environment is important because agent design depends on it.

8.1 Fully Observable vs Partially Observable

Fully Observable Environment

The agent can perceive the complete state of the environment at any time.

Example

Chess

Both players can see the whole board.

Partially Observable Environment

The agent cannot see the complete environment.

Example

Driving a car in fog

The driver cannot see everything clearly.

8.2 Deterministic vs Stochastic

Deterministic Environment

The next state is completely determined by the current state and the agent's action.

Example

Solving a math problem

Same input gives same result.

Stochastic Environment

The next state is uncertain and may involve randomness.

Example

Self-driving car in traffic

Other drivers may behave unpredictably.

8.3 Episodic vs Sequential

Episodic Environment

An environment where each decision (episode) is independent of previous decisions.
The current action does not depend on past actions.

Think: One task → Done → Next task starts fresh

Example

Medical Test Result Classification

Situation:

A system analyzes X-ray or blood test results.

For each patient:

Disease

No Disease

Each patient's test is evaluated independently.

Why this is Episodic:

The diagnosis of one patient does not affect another patient.

Sequential Environment

Current decisions affect future decisions.

Example

Chess

One move affects all later moves.

8.4 Static vs Dynamic

Static Environment

The environment does not change while the agent is thinking.

Example

Crossword puzzle

Dynamic Environment

The environment changes over time.

Example

Driving in traffic

Other cars keep moving.

8.5 Discrete vs Continuous

Discrete Environment

An environment that has a **limited number of clearly separated states or actions**.

Example

Chess

Moves and positions are discrete.

Continuous Environment

An environment where **states or actions can take any value within a range**, not just fixed numbers.

Example

Controlling the speed of a car

Speed can vary continuously.

8.6 Single-Agent vs Multi-Agent

Single-Agent Environment

Only one agent is operating.

Example

A person solving a Sudoku puzzle

Multi-Agent Environment

More than one agent interacts in the environment.

Example

Football match, traffic system, online gaming

9. Summary Table of Environment Types

Environment Property	Type 1	Type 2	Example
Observability	Fully Observable	Partially Observable	Chess / Driving in fog
Outcome	Deterministic	Stochastic	Math problem / Traffic
Decision Dependency	Episodic	Sequential	Image classification / Chess
Change over time	Static	Dynamic	Crossword / Road traffic
State values	Discrete	Continuous	Chess / Car speed
Number of agents	Single-Agent	Multi-Agent	Sudoku / Football

10. Agent Architecture

Agent architecture refers to the structure or framework used to build an agent.

It defines how the agent:

1. receives percepts
2. processes information
3. selects actions

In simple words, architecture is the “system design” of the agent.

General Form

Agent = **Architecture** + **Program**

Explanation

- ★ **Architecture**: the machine or platform
- ★ **Program**: the logic running on it

Example

For a robot:

- **Architecture** = sensors, motors, hardware
- **program** = AI algorithm controlling movement

Types of Agent Architectures

1. Simple Reflex Architecture

Uses condition-action rules.

Example:

→ if obstacle ahead → turn right

Best for simple environments.

2. Model-Based Architecture

Uses an internal state or memory.

Best for **partially observable** environments.

3. Goal-Based Architecture

Includes goals and planning.

Best when the agent must reach a **specific target**.

4. Utility-Based Architecture

Use a utility function to compare options.

Best for choosing among multiple good solutions.

5. Learning Architecture

Improves through feedback and experience.

Best for modern intelligent systems.

Real-World Understanding of Agent Architecture

Example: Food Delivery Robot

Architecture

1. camera
2. proximity sensors
3. wheels
4. battery
5. GPS

Program

1. detect path
2. avoid obstacles
3. locate delivery address
4. optimize route

Without architecture, the program cannot act.

Without program, the hardware cannot behave intelligently.

Both are needed together.

Example 1: Thermostat

A thermostat can be treated as a simple reflex agent.

Rule:

- if temperature $<$ set point $\rightarrow$ heater on
- if temperature $\geq$ set point $\rightarrow$ heater off

Example 2: ATM Machine

ATM acts like an agent:

- sensors: keypad, card reader
- actuators: cash dispenser, screen
- performance measure: correct transaction, security
- environment: user, bank database

Example 3: Mobile Navigation App

- sensors: GPS, map data, traffic data
- actuators: voice directions, route display
- goal: reach destination
- utility: shortest or fastest route

Lab Session: Simple Rule-Based Agent in Python

A **rule-based agent** works using simple if-else rules.

Lab Objective

Build a small AI agent that decides an action based on the condition of a room.

Problem Statement

Design a vacuum cleaner agent for two rooms:

1. Room A
2. Room B

The agent should:

1. clean if the room is dirty
2. move to the other room if the current room is clean

Python Code

```
def vacuum_agent(location, status):
    if status == "Dirty":
        return "Suck"
    elif location == "A":
        return "Move Right"
    elif location == "B":
        return "Move Left"

# Test cases
print("Agent in Room A, Dirty:", vacuum_agent("A", "Dirty"))
print("Agent in Room A, Clean:", vacuum_agent("A", "Clean"))
print("Agent in Room B, Dirty:", vacuum_agent("B", "Dirty"))
print("Agent in Room B, Clean:", vacuum_agent("B", "Clean"))
```

Output

Agent in Room A, Dirty: Suck
Agent in Room A, Clean: Move Right
Agent in Room B, Dirty: Suck

Agent in Room B, Clean: Move Left

Explanation of the Code

This agent follows very basic rules:

1. if the current room is dirty, it cleans the room
2. if the current room is clean and it is in Room A, it moves right
3. if the current room is clean and it is in Room B, it moves left

This is a **simple reflex agent** because:

1. it only uses the current input
2. it does not remember the past
3. it uses condition-action rules

2nd example

```
def vacuum_agent(location, status):  
    if status.lower() == "dirty":  
        return "Suck"  
    elif location.upper() == "A":  
        return "Move Right"  
    elif location.upper() == "B":  
        return "Move Left"  
    else:  
        return "Invalid Input"
```

```
location = input("Enter current room (A/B): ")  
status = input("Enter room status (Dirty/Clean): ")
```

```
action = vacuum_agent(location, status)  
print("Agent Action:", action)
```

15. Another Rule-Based Agent Example: Weather Agent

This example is even easier for students.

```
def weather_agent(weather):
    if weather.lower() == "rainy":
        return "Take an umbrella"
    elif weather.lower() == "sunny":
        return "Wear sunglasses"
    elif weather.lower() == "cold":
        return "Wear a jacket"
    else:
        return "Stay prepared"

# Test
weather = input("Enter weather condition: ")
print("Agent Decision:", weather_agent(weather))
```

Explanation

This is also a rule-based agent because it maps a condition directly to an action.

Problem Solving & Search Algorithms

Breadth First Search (BFS)

Introduction

Breadth First Search (BFS) is an **uninformed search algorithm** used in Artificial Intelligence to search trees, graphs, and state spaces.

It explores nodes **level by level**, meaning it visits all neighboring nodes first before moving to deeper levels.

BFS is called an uninformed or blind search because it does not use any additional knowledge about the problem.

Core Idea of BFS

The main idea of BFS is:

“Expand the shallowest unexpanded node first.”

The search proceeds breadth-wise through the search tree.

Working of BFS

BFS works in the following way:

1. Start from the initial node.
2. Visit all neighboring nodes.
3. Store discovered nodes for future exploration.
4. Expand nodes in the order they were discovered.
5. Continue until the goal state is found.

Data Structure Used

BFS uses a:

Queue (FIFO)

FIFO means:

- ❖ First In
- ❖ First Out

This helps BFS explore nodes level by level.

BFS in AI

In AI, BFS is used to search through a **state space** where:

- ★ nodes represent states
- ★ edges represent actions

It systematically explores possible states until it reaches the goal state.

Properties of BFS

1. Completeness

BFS is complete, meaning:

- ♦ if a solution exists,
- ♦ BFS will find it.

2. Optimality

BFS is optimal when:

- ★ all step costs are equal.

It finds the shortest path in terms of number of steps.

Advantages of BFS

- ☺ Complete algorithm
- ☺ Finds shortest path
- ☺ Simple to understand
- ☺ Systematic exploration

Disadvantages of BFS

- ⊗ High memory usage
- ⊗ Slow for deep search spaces
- ⊗ State space may grow exponentially

Applications of BFS in AI

- Shortest path finding
- Robot navigation
- Maze solving
- Web crawling
- Social network analysis

Depth First Search (DFS)

Introduction

Depth First Search (DFS) is an **uninformed search algorithm** used in Artificial Intelligence to search trees, graphs, and state spaces.

DFS explores nodes by going **deep into one branch first** before backtracking and exploring other branches.

It is called an uninformed or blind search because it does not use additional knowledge about the problem.

Core Idea of DFS

The main idea of DFS is:

“Expand the deepest unexpanded node first.”

DFS continues moving deeper until:

- ✓ the goal is found, or
- ✓ no further nodes exist.

Then it backtracks to explore another path.

Working of DFS

DFS works in the following way:

1. Start from the initial node.
2. Move to one child node.
3. Continue exploring deeper nodes.
4. If no further node exists, backtrack.
5. Continue until the goal state is found.

Data Structure Used

DFS uses a:

Stack (LIFO)

LIFO means:

- ♦ Last In
- ♦ First Out

The most recently added node is explored first.

DFS in AI

In AI, DFS searches through a **state space** where:

- nodes represent states
- edges represent actions

DFS explores one possible solution path completely before trying another path.

Commented [AR1]: A **state space** is the collection of **all possible states or situations** that can occur in a problem.
In AI:
each state represents a condition of the problem
moving from one state to another is done through actions

Properties of DFS

1. Completeness

DFS is **not always complete**.

If the search space contains infinite depth or loops, DFS may never find the solution.

2. Optimality

DFS is **not optimal**.

It does not guarantee the shortest path or best solution.

DFS requires less memory than BFS because it stores only the current path.

Advantages of DFS

- ☺ Requires less memory
- ☺ Simple to implement
- ☺ Useful for deep search problems

Disadvantages of DFS

- ★ Not guaranteed to find solution
- ★ May get stuck in deep or infinite paths
- ★ Does not guarantee shortest path

Applications of DFS in AI

- ★ Puzzle solving
- ★ Game tree search
- ★ Maze exploration
- ★ Path finding
- ★ Backtracking problems

BFS vs DFS

Feature	BFS	DFS
Search Style	Level-wise	Depth-wise
Data Structure	Queue	Stack
Complete	Yes	No
Optimal	Yes	No
Memory Usage	High	Low

Uniform Cost Search (UCS)

Introduction

Uniform Cost Search (UCS) is an **uninformed search algorithm** used in Artificial Intelligence to find the path with the **lowest total cost**.

Unlike BFS, UCS does not expand nodes level by level.

Instead, it expands the node with the **smallest path cost first**.

UCS is useful when:

- different actions have different costs
- the cheapest path is required

Core Idea of UCS

The main idea of UCS is:

“Expand the node with the lowest cumulative cost.”

The algorithm always chooses the cheapest path available at that moment.

Working of UCS

UCS works in the following way:

1. Start from the initial node.
2. Calculate path cost for each node.
3. Expand the node with the smallest cost.
4. Continue updating costs while searching.
5. Stop when the goal state is reached.

Data Structure Used

UCS uses a:

Priority Queue

Nodes are stored according to their path cost.

- lower cost = higher priority
- higher cost = lower priority

UCS in AI

In AI, UCS searches through a **state space** where:

- ✓ nodes represent states
- ✓ edges represent actions with costs

The algorithm always explores the least expensive path first.

Properties of UCS

1. Completeness

UCS is complete, meaning:

- ★ if a solution exists,
- ★ UCS will find it.

2. Optimality

UCS is optimal.

It always finds the minimum-cost path.

Advantages of UCS

- Finds lowest-cost solution
- Complete algorithm
- Optimal algorithm
- Works well with varying path costs

Disadvantages of UCS

- ◆ High memory usage
- ◆ Can be slow for large state spaces
- ◆ Explores many unnecessary nodes

Applications of UCS in AI

- ☺ GPS navigation systems
- ☺ Network routing
- ☺ Robot path planning
- ☺ Transportation systems
- ☺ Cost optimization problems

BFS vs UCS

Feature	BFS	UCS
Expansion Rule	Shallowest node first	Lowest-cost node first
Edge Costs	Equal costs	Different costs
Optimal	Yes (equal costs only)	Yes
Data Structure	Queue	Priority Queue